## Scale

# NestUp

Heavy queries, indexes, caching, measured results, and limits

**Internet Technologies — Become a Full-Stack Engineer**

RUNI CS 2026 · Final project

Live product **nestup-kappa.vercel.app**

Repository **github.com/nestupweb/nestup**

**Current production data:** 845 profiles · 824 listings · 1,290 household memberships · 883 listing views · 29 conversations **Database:** Postgres 17, Supabase (eu-central-1) · **Hosting:** Vercel serverless

---

## 1. Where the product is today

Worth stating first, because it changes what the honest answers below are: this is not a toy dataset. The seed is 838 accounts and 824 live rooms across 124 cities — roughly the size of a real product's first city. So the queries described here are running against realistic row counts, not three test records, and the measurements are from the live deployment.

---

## 2. What happens as users arrive

### Tens of concurrent users

No problem, and no change needed. Vercel runs each request in its own serverless invocation, so concurrency is handled by the platform rather than by a process pool. The database sits well inside Supabase's pooled connection limits. The heaviest read — the Listings index — is a **shared cache entry** served identically to everyone.

### Hundreds of concurrent users

Still fine, with one caveat worth naming.

What holds up well:

- **The public room list scales with content, not traffic.** `queryListings` is a shared `"use cache"` keyed on the filter set, so a thousand people opening Listings with default filters cause **one** database query, not a thousand.
- **Per-member data never touches a shared store.** Decks, profile tabs, saved ids and inboxes are `"use cache: private"` — held in each member's own browser. They cost nothing on the server and cannot contend.
- **Pages ship HTML, not a client-side fetch waterfall.**

What would show strain first:

- **Realtime connections.** Every open chat holds a WebSocket. Supabase's free tier caps concurrent connections, and this is the first ceiling the product would hit — before CPU, before storage, before query time.
- **Deck construction.** See §3 — it is the one genuinely expensive read, and it cannot be shared between members.

### Thousands

Requires the work in §8. Chiefly: the deck must stop scoring in application code, and Realtime needs a plan.

---

## 3. Heavy queries, and what was done about them

### 3.1 The swipe deck — by far the most expensive

Building one member's deck was originally **~9 round-trips in 3 dependent waves**: candidate rooms, then owners and residents, then blocked ids and attention history. Every one of them is scoped to a single member, so nothing can be shared.

Three things keep it acceptable:

1. **Hard filters run in SQL, before scoring.** The city filter, `is_active`, `removed_at is null` and the already-swiped exclusion all happen in Postgres. Only survivors are scored. Scoring 824 rooms in Node for every deck view would be the naive version of this.
2. **A deck cap.** `DECK_SIZE = 60`.
3. **The whole result is cached per member** (`getCachedDeck`, tagged `deck:<userId>`), invalidated only when the member swipes or changes a room of their own.

**Honest limitation:** the scoring itself is JavaScript over the filtered candidate set. That is fine at 824 rooms in one city; at 50,000 it is not, and §8 says what replaces it.

### 3.2 The Listings index

`select * from listings where is_active` plus filters, with `count: 'exact'` for pagination.

- Backed by `listings_browse_idx on (city, rent, available_from) where is_active` — a **partial** index, so it only covers rows that can actually appear.
- Paginated at 20, and the page is applied in SQL via `range()`, never in JavaScript.
- Shared-cached, so repeat traffic is free.

**Honest limitation:** `count: 'exact'` makes Postgres count every matching row on every uncached query, which degrades as the table grows. An estimated count is the standard fix, deferred because "824 rooms available" being exactly right is worth more than the milliseconds at this size.

### 3.3 The map — all pins at once

`/api/listings/pins` returns **every** placed room (~824), about 150 KB. That is a deliberate product decision: the map is meant to show everything, not the current filter.

The cost is contained by only paying it when it is needed:

- The endpoint selects **only** `id, lat, lng, rent, title, city, neighborhood, photo_urls[0]` — not `select *`.
- **No map renders until an icon is pressed.** MapLibre and the pin payload are both loaded on first open, and kept for the rest of the visit. Most visits never open a map and never pay for it.

### 3.4 The chat inbox

`my_conversations()` — one SQL function returning every thread with its last message, unread count and the other party. One round-trip, not one query per conversation.

Backed by `messages_conversation_idx (conversation_id, created_at)`.

### 3.5 Household size and gender

Needed on every row of the browse list and every deck candidate. Computing them per row would turn one query into hundreds, so they are **denormalised onto `listings`** and maintained by triggers over `listing_residents`. Classic write-cost-for-read-benefit trade, and the right way round here: rooms are read far more often than households change.

---

## 4. Indexes

**20 indexes**, each placed against a query that exists. The ones that carry the product:

| INDEX | SERVES |
| --- | --- |
| `listings_browse_idx (city, rent, available_from) WHERE is_active` | The Listings index and every deck's hard filter. Partial — inactive rooms are not in it |
| `one_active_listing_per_owner (owner_id) WHERE is_active` | **Unique.** Enforces "one person, one home" as a constraint, not a check |
| `swipes_by_seeker_idx (seeker_id)` | Excluding already-swiped rooms — read on every deck build |
| `swipes_likes_by_listing_idx (listing_id) WHERE direction = 'like'` | Interest counts per room. Partial: skips are the majority and are never counted |
| `messages_conversation_idx (conversation_id, created_at)` | Thread history and last-message lookups |
| `messages_client_id_uniq` | **Unique.** Idempotent retry — the reason a resend cannot double-post |
| `listing_views_recent_idx (user_id, viewed_at DESC)` | Profile › History, `LIMIT 30`. `DESC` matches the query's order |
| `viewings_by_conversation_idx (conversation_id, created_at)` | Viewings inside a thread |
| `saved_listings_by_listing_idx (listing_id)` | Save counts |
| `blocks_blocked_idx (blocked_id)` | `blocked_user_ids()` — called on every deck build |
| `listing_invites_pending_idx (invitee_id) WHERE status = 'pending'` | Pending invitations. Partial: answered invites are dead weight |
| `profiles_full_name_trgm_idx` | **Trigram** index for roommate-name search — a `LIKE '%name%'` cannot use a B-tree |
| `listings_household_size_idx`, `listings_household_gender_idx`, `listings_wanted_gender_idx` | The denormalised filter columns |

The pattern worth pointing at: **six of these are partial or expression indexes.** A partial index on `WHERE is_active` is smaller and stays hot in cache, because it never contains the rows the query cannot return anyway.

---

## 5. Avoiding unnecessary loading

This is where most of the recent engineering went, and it is measurable.

### 5.1 Server-side caching

Five cached reads, each tagged so a write invalidates exactly what it changed:

| READ | CACHE | TAG | INVALIDATED BY |
|---|---|---|---|
| `queryListings` | **shared** | `listings` | Publishing, editing, pausing, removing a room |
| `getSavedListingIds` | private | `saved:<id>` | Hearting |
| `getProfileTabData` | private | `profile:<id>` | Profile/listing/heart/invite changes |
| `getCachedDeck` | private | `deck:<id>` | Swiping; own-room changes |
| `getCachedInbox` | private | `chat:<id>` | Sending, reading, deleting a chat; **and the other side's message, via Realtime** |

That last row is the interesting one. A member's inbox changes when *someone else* writes, and no action of theirs runs. The Realtime socket calls a Server Action that drops their own tag — which is what makes caching an inbox safe rather than stale.

**The blast-radius rule.** Mutations used to answer every write with a fistful of `revalidatePath` calls, so editing a room threw away the member's chats and profile too. Now a write invalidates only what it changed, and `cache-invalidation.test.ts` fails if that regresses.

### 5.2 Measured result

Returning to an already-visited tab, on production, warm cache:

| TAB | BEFORE | AFTER |
|---|---|---|
| Swipe | 860 ms, skeleton | **69 ms, no skeleton** |
| Chat | 1,900 ms, skeleton | **71 ms, no skeleton** |
| Profile | ~370 ms, skeleton | **69 ms, no skeleton** |
| Listings | ~850 ms, skeleton | ~375 ms, ~300 ms skeleton |

Three of four tabs make **zero server requests** on return.

The technical reason is worth being able to explain: Next's App Shell prerender advances through cached reads and **stops at the first uncached one**. Every page began with an uncached `auth.getUser()` — a network round-trip to Supabase — which kept everything behind it out of the shell no matter how well cached it was. Caching that identity read is what removed ~300 ms from each tab.

Listings is the remaining case. It makes no server request either; its residual is ~60 ms of client render, stretched by the page-slide animation. That is an animation-timing issue, not a data one.

### 5.3 Payload discipline

- **Pagination everywhere it matters** — Listings 20/page in SQL, History `LIMIT 30`, deck capped at 60.
- **Column selection** — the pins endpoint selects 8 columns, not `*`.
- **Images** — `next/image` with correct `sizes`, so a 128 px thumbnail is not a 1200 px file. Only the first three covers are `priority`; the swipe deck warms exactly **one** card ahead, not the whole deck.
- **Code splitting** — MapLibre is a dynamic import behind a button press.
- **Photos are compressed in the browser** before upload.

---

## 6. Client / server separation

| RUNS ON THE SERVER | RUNS IN THE BROWSER |
|---|---|
| Every database read (Server Components) | Interaction state — dialogs, the current card, filter drafts |
| Every database write (Server Actions) | Optimistic message rendering |
| All scoring and ranking | Map rendering (MapLibre/WebGL) |
| All authorisation | Image compression before upload |
| Secrets — service-role key, SMTP, Gemini | Realtime subscription |

**No credential capable of bypassing RLS ever reaches the browser.** The client holds the anon key only, which is designed to be public and is powerless without a session. The service-role key exists solely in server-side environment variables.

Two boundary details that are enforced rather than assumed:

- `lib/supabase/public.ts` is a **cookie-free** client used only for the shared cache. A session-bearing client there would make the shared entry member-specific — a cross-user leak — so it is `server-only` and documented as such.
- `lib/swipe.ts` (client-safe) and `lib/swipe-deck.ts` (`server-only`) are split precisely because the cookie-reading client must not enter a browser bundle.

---

## 7. Current limitations

Stated honestly; each is a real ceiling.

1. **Deck scoring is application-side.** Fine at ~824 rooms; the wrong shape at 50,000.
2. **`count: 'exact'`** on the Listings query counts every matching row on a cache miss.

3. **Realtime connection limits.** The first hard ceiling on the free tier.
4. **No rate limiting except on auth email.** `auth_mail_throttle` protects the mail path; message sending and listing creation are protected only by RLS.
5. **Photos are checked by an external model on the upload path**, which adds latency and depends on a third party's availability.
6. **No background jobs.** Everything happens in the request. Notification emails are sent inline.
7. **Single region.** eu-central-1 for the database; fine for an Israel-focused product, poor for anyone else.
8. **No CDN caching of the pins payload.** 150 KB is fetched per visitor who opens the map.
9. **Storage is unbounded.** Nothing prunes photos of removed listings.
10. **No observability.** No metrics, no tracing, no slow-query alerting. Problems would be discovered by a person noticing.

---

## 8. What a larger version would change

In the order the pain would actually arrive.

**First — measure.** Nothing here is worth doing before there is a slow-query log and a p95 latency number per route. Every item below is a hypothesis until then.

1. **Move deck filtering fully into SQL.** Compute the weighted score as a SQL expression (or a materialised per-member candidate table refreshed on profile change), so Postgres returns 60 ranked rows instead of Node scoring hundreds. This is the single change that unlocks an order of magnitude.
2. **Estimated counts.** Swap `count: 'exact'` for a planner estimate above a threshold, keeping exact counts for small result sets.
3. **Cursor pagination.** `range()` offsets degrade deep into a list; keyset pagination on `(created_at, id)` does not.
4. **Move notification email to a queue.** It should not be on the request path.
5. **Rate limiting** on message send, listing creation and report submission.
6. **Cache the pins payload at the CDN edge** with a tag-based purge — it is identical for everyone.
7. **Realtime plan:** move to a paid tier, or replace always-on sockets with polling for the inbox and reserve sockets for open threads.
8. **Add observability** — Vercel Analytics, Supabase slow-query logs, an error tracker.
9. **Storage lifecycle** — delete photos when a listing is hard-removed.
10. **Read replicas / multi-region** only if the product ever leaves one country. Listed last on purpose: it is the change people reach for first and need last.

---